

Object Oriented Programming (Inheritance)

Abbas Khalid

April 21, 2025

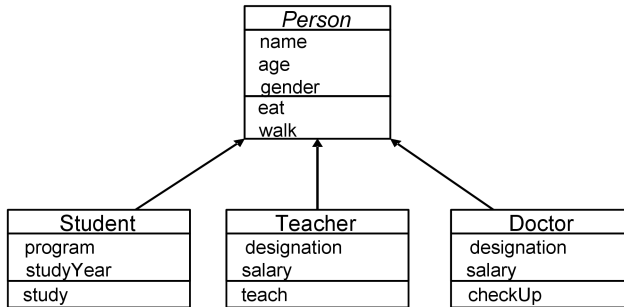
Table of contents

- 1 Inheritance
- 2 Single Inheritance
- 3 Multi-level Inheritance

Inheritance

- While creating a class, instead of writing completely new data members and member functions, the programmer can designate that new class should inherit the members of an existing class.
- This saves time during program development and encourages the reuse of debugged high-quality code. This process is known as **Inheritance**.
- The existing class is known as **base/ parent/ super** class and the new class is referred to as **derived/ child/ sub** class. Typically, a derived class contains behavior inherited from its base class plus its own.
- Classes derived from others inherit all the visible members of the base class e.g. if a base class includes a member *A* and we derive it to another class with another member called *B*, the derived class will contain both *A* and *B*.

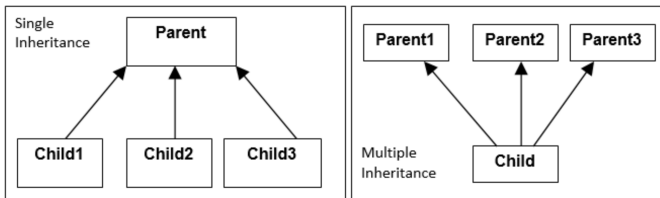
Inheritance



Due to Inheritance, all the members of **Person** become the members of derived classes as well.

Single and Multiple Inheritance

- C++ supports two types of inheritances.
- In **single inheritance**, a class is derived from one base class.
- In **multiple inheritance**, a class is derived from multiple classes.



Protected Members

- Using *protected* access offers an intermediate level of protection between *public* and *private* access.
- A base class's *protected* members can be accessed by members and friends of that base class.
- A base class's *protected* members can also be accessed by members and friends of the classes derived from that base class.
- Derived class member functions can refer to *protected* members of the base class simply by using the member names (just like *public* members).
- When the same function exists in both the base class and the derived class, the function in the derived class will be executed. We say that the derived class function overrides the base class function.

When a derived class re-defines a base class member, the base class member can be accessed from the derived class by preceding the base class member name with the base class name and the binary scope resolution operator (::).

className :: memberName;

Example

```
1 class P{
2     protected:
3         int x;
4     public:
5         void set(int a){x = a;}
6         int get(){return x;}
7 };
8
9 int main() {
10
11     P obj;
12     // obj.x = 10;           //Error! x is protected
13     obj.set(10);           //Only public members can be accessed
14     cout<<"The value is: "<<obj.get()<<endl;
15
16     return 0;
17 }
```

The value is: 10

Single Inheritance

- In order to derive a class from another, we must use the operator : (colon) in the declaration of the derived class in the following way:

```
1 class Child : mode Parent {  
2 // body of Child Class  
3 };
```

where *Child* is the name of the derived class and *Parent* is the name of the class on which it is based. The word *mode* may be replaced by any of the access specifiers like *public*, *protected* or *private*, and describes the mode of inheritance

- If no access specifier is present, the access specifier is *private* by default if the derived class is a *class*. If the derived class is a *struct*, then *public* is the default.

Public Inheritance

- With *public* inheritance, every object of a derived class is also an object of that derived class's base class. However, base class objects are not objects of their derived classes.
- For example, all cars are vehicles but not all vehicles are cars. Because one base class can have many derived classes, the set of objects represented by base class typically is larger than the set of objects represented by any of its derived classes. In public inheritance:
 - Public members are inherited to derived class with no change in access level. These can be accessed directly by non-*static* member functions, *friend* functions and non-member functions.
 - Protected members are inherited to derived class with no change in access level. These can be accessed directly by non-*static* member functions and *friend* functions.
 - Private members are hidden in the derived class. They can be accessed by non-*static* member functions and *friend* functions through *public* or *protected* member functions of the base class.

Example 1

```
1 class Parent {
2     public:
3         int x, y;
4         void set (int a, int b){ x = a; y = b;}
5 };
6
7 class Child : public Parent {
8     public:
9         int sum (){ return x+y; }
10 };
11 int main () {
12     Child c;
13     c.set (4,5);
14
15     /* set is inherited to Child class and is public, so can be accessed from
16        the main i.e. (outside the parent and the child class). Similarly,
17        c.x = 4; and c.y = 5; are valid. */
18
19     cout<<" Sum: " << c.sum() << endl;
20     return 0;
21 }
```

Sum is: 9

Example 2

```
1 class Parent {
2     protected:
3         int x, y;
4         void set (int a, int b){ x = a; y = b;}
5 };
6
7 class Child : public Parent {
8     public:
9         int sum (){ return x+y; }
10 };
11 int main () {
12     Child c;
13     c.set (4,5);    // Error!
14
15     /* set is inherited to Child class and is protected, so can't be accessed
16        from the main i.e. (outside the parent and the child class).
17        Similarly, c.x = 4; and c.y = 5; are invalid. */
18
19     cout<<" Sum: " << c.sum() << endl;
20     return 0;
21 }
```

Example 3

```
1 class Parent {
2     protected:
3         int x, y;
4     public:
5         void set (int a, int b){ x = a; y = b;}
6 };
7 class Child : public Parent {
8     public:
9         int sum (){ return x+y; }
10 };
11 int main () {
12     Child c;
13     c.set (4,5);
14
15     /* set is inherited to Child class and is public, so can be accessed from
16        the main i.e. (outside the parent and the child class). However, c.x
17        = 4; and c.y = 5; would be invalid. */
18
19     cout<<" Sum: " << c.sum() << endl;
20     return 0;
21 }
```

Sum is: 9

Example 4

```
1 class Parent {
2     int x;
3     public:
4         void display_Parent(){cout<<"Parent Class:"<<x<<endl;}
5         void set (int a){x = a;}
6 };
7
8 class Child : public Parent {
9     public:
10        void display_Child(){cout<<"Child Class. " <<endl;}
11 };
12
13 int main () {
14     Child c;    c.set(100);
15     c.display_Parent();
16     return 0;
17 }
```

Parent Class: 100

Example 5

```
1 class Parent {
2     int x;
3     public:
4         void set (int a){x = a;}
5         int get () {return x;}
6 };
7
8 class Child : public Parent {
9
10 };
11
12 int main () {
13
14     Child c;    c.set(100);
15     cout<<"Parent Class Private Member:"<<c.get()<<endl;
16
17     return 0;
18 }
```

Parent Class Private Member: 100

Example 6

```
1 class Parent {
2     int x;
3     public:
4         void display_Parent(){cout<<"Parent Class:"<<x<<endl;}
5         void set (int a){x = a;}
6 };
7
8 class Child : public Parent {
9     int x;
10    public:
11        void display_Child(){cout<<"Child Class:"<<x<<endl;}
12 };
13
14 int main () {
15
16     Child c;    c.set(10);
17     c.display_Parent();
18     c.display_Child();
19
20     return 0;
21 }
```

Parent Class: 10
Child Class: 0

Example 7

```
1 class Parent {
2     int x;
3     public:
4         void display_Parent(){cout<<"Parent Class:"<<x<<endl;}
5 };
6
7 class Child : public Parent {
8     int x;
9     public:
10        void display_Child(){cout<<"Child Class:"<<x<<endl;}
11        void set (int a){x = a;}
12 };
13
14 int main () {
15
16     Child c;    c.set(10);
17     c.display_Parent();
18     c.display_Child();
19
20     return 0;
21 }
```

Parent Class: 0
Child Class: 10

Example 8

```
1 class Parent {
2     int x;
3     protected:
4     int y;
5     public:
6     void set (int a, int b){ x = a; y = b;}
7 };
8
9 class Child : public Parent {
10     public:
11     int sum (){ return x+y; }
12 };
13 int main () {
14     Child c;
15     c.set (4,5);    // OK
16     /* set is inherited to Child class and is public, y is inherited to Child
17        class and is protected, x is not inherited to Child class. */
18
19     cout<<" Sum: " << c.sum() << endl; // Error!
20 return 0;
21 }
```

Example 9

```
1 class Parent {
2     public:
3         void display(){cout<<"Parent Class."<<endl;}
4 };
5
6 class Child : public Parent {
7     public:
8         void display(){cout<<"Child Class."<<endl;}
9 };
10
11 int main () {
12
13     Child c;    c.display();
14
15     return 0;
16 }
```

Child Class.

Example 10

```
1 class Parent {
2     public:
3         void display(){cout<<"Parent Class."<<endl;}
4 };
5
6 class Child : public Parent {
7     public:
8         void display(){Parent::display();}
9 };
10
11 int main () {
12
13     Child c;    c.display();
14
15     return 0;
16 }
```

Parent Class.

Example 11

```
1 class Parent {
2     public:
3         void display(){cout<<"Parent Class."<<endl;}
4 };
5
6 class Child : public Parent {
7     public:
8         void display(){cout<<"Child Class."<<endl;}
9 };
10
11 int main () {
12
13     Child c;    c.display();
14     c.Parent::display();
15
16     return 0;
17 }
```

Child Class.
Parent Class.

Protected and Private Inheritance

- When the base class is inherited by using the *protected* access specifier, all *public* and *protected* members of the base class become *protected* members of the derived class.
- When the base class is inherited by using the *private* access specifier, all *public* and *protected* members of the base class become *private* members of the derived class.

Example 1

```
1 class Parent {
2     public:
3         int x, y;
4         void set (int a, int b){ x = a; y = b;}
5 };
6
7 class Child : protected Parent {
8     public:
9         int sum (){ return x+y; }
10 };
11 int main () {
12     Child c;
13     c.set (4,5);    // Error!
14
15     /* set is inherited to Child class and is protected, so can't be accessed
16        from the main i.e. (outside the parent and the child class).    */
17
18     cout<<" Sum: " << c.sum() << endl; // OK
19 return 0;
20 }
```

Example 2

```
1 class Parent {
2     public:
3         int x, y;
4         void set (int a, int b){ x = a; y = b;}
5 };
6
7 class Child : private Parent {
8     public:
9         int sum (){ return x+y; }
10 };
11 int main () {
12     Child c;
13     c.set (4,5);    // Error!
14
15     /* set is inherited to Child class and is private, so can't be accessed
16        outside the Child class.    */
17
18     cout<<" Sum: " << c.sum() << endl; // OK
19 return 0;
20 }
```

Example 3

```
1  class Parent {
2      public:
3          int x, y;
4          void set (int a, int b){ x = a; y = b;}
5  };
6
7  class Child : Parent {
8      public:
9          int sum (){ return x+y; }
10 };
11 int main () {
12     Child c;
13     c.set (4,5);    // Error!
14
15     /* set is inherited to Child class and is private, so can't be accessed
16        outside the Child class.    */
17
18     cout<<" Sum: " << c.sum() << endl; //OK
19     return 0;
20 }
```

Constructors, Destructors, and Inheritance

- Constructors are not inherited. Each class must define its own constructors. However, derived classes can call base class constructors to initialize the base part of the object. Destructors are inherited. However, if a destructor is explicitly defined in the derived class, it overrides the inherited destructor.
- When an object of a derived class is created, if the base class contains a constructor, it will be called first, followed by the derived class' constructor.
- When a derived object is destroyed, its destructor is called first, followed by the base class' destructor, if it exists.
- Arguments to the base class constructor can be passed through the constructor of the derived class as follows.

```
1 Child (parameterList):Parent (argumentList) {  
2     // body of Child Class Constructor  
3 }
```

Here, the *argumentList* must be consistent with the parameter list of the constructor defined for the base class.

Example 1

```
1 class Parent {
2     public:
3         Parent(){ cout<<"Parent Class Constructor.\n";}
4         ~Parent(){ cout<<"Parent Class Destructor.\n";}
5 };
6
7 class Child : public Parent {
8     public:
9         Child(){ cout<<"Child Class Constructor.\n";}
10        ~Child(){ cout<<"Child Class Destructor.\n";}
11 };
12
13 int main () {
14
15     Child c;
16
17     return 0;
18 }
```

Parent Class Constructor.
Child Class Constructor.
Child Class Destructor.
Parent Class Destructor.

Example 2

```
1 class Parent {
2     protected:
3         int x;
4     public:
5         Parent(int a){ x = a; cout<<"Parent Constructor.\n";}
6         ~Parent(){ cout<<"Parent Destructor.\n";}
7 };
8
9 class Child : public Parent {
10     int y;
11     public:
12         Child(int i, int j):Parent(j)
13             {y = i; cout<<"Child Constructor.\n";}
14         ~Child(){ cout<<"Child Destructor.\n";}
15         void display(){cout<<x<<" and "<<y<<endl;}
16 };
17
18 int main () {
19     Child c (5,10);    c.display();
20     return 0;
21 }
```

Parent Constructor.
Child Constructor.
10 and 5
Child Destructor.
Parent Destructor.

Scenario 8.1

```
1 class Person {
2     string name;
3     int age;
4
5     public:
6         Person(string name, int age) : name(name), age(age) {}
7
8         void setName(string name) { this->name = name; }
9         void setAge(int age) { this->age = age; }
10
11        string getName() const { return name; }
12        int getAge() const { return age; }
13
14        void printInfo() const {
15            cout << "Name: " << name << "\n";
16            cout << "Age: " << age << "\n";
17        }
18 };
```

Scenario 8.1 (Cont.)

```
1 class Student : public Person {
2     int studentId;
3     string grade;
4
5     public:
6         Student(string name, int age, int Id, string grade)
7             : Person(name, age), studentId(Id), grade(grade) {}
8
9         void setStudentId(int Id) { studentId = Id; }
10        void setGrade(string grade) { this->grade = grade; }
11
12        int getStudentId() const { return studentId; }
13        string getGrade() const { return grade; }
14
15        void printInfo() const {
16            Person::printInfo(); // Call Person's method
17            cout << "Student ID: " << studentId << "\n";
18            cout << "Grade: " << grade << "\n";
19        }
20 };
```

Scenario 8.1 (Cont.)

```
1 int main() {  
2  
3     Student S("Imran Ali", 25, 8288151, "A");  
4     S.printInfo();  
5  
6     return 0;  
7 }
```

```
Name: Imran Ali  
Age: 25  
Student ID: 8288151  
Grade: A
```

Multi-level Inheritance

- When a derived class is used as a base class for another derived class, any protected member of the initial base class that is inherited (as public) by the first derived class may also be inherited as protected again by a second derived class.
- When a derived class object is created, the base class constructor must be called before the derived class constructor.
- When several levels of inheritance are involved, the class constructor of the most general base class is called first; subsequent constructor calls are made in order of derivation, until finally the derived class constructor is called.

Example 1

```
1 class Parent {
2     public:
3         void message(){cout<<"Parent Class."<<endl;}
4 };
5
6 class Child : public Parent {
7     public:
8         void message(){cout<<"Child Class."<<endl;}
9 };
10
11 class GrandChild : public Child{
12     public:
13         void message(){cout<<"Grand Child Class."<<endl;}
14 };
15
16 int main(){
17     GrandChild GC; GC.message();
18     return 0;
19 }
```

Grand Child Class.

Example 2

```
1 class Parent {
2     public:
3         void message(){cout<<"Parent Class."<<endl;}
4 };
5
6 class Child : public Parent {
7     public:
8         void message(){cout<<"Child Class."<<endl;}
9 };
10
11 class GrandChild : public Child{
12
13 };
14
15 int main(){
16     GrandChild GC;  GC.message();
17     return 0;
18 }
```

Child Class.

Example 3

```
1 class Parent {
2     public:
3         void message(){cout<<"Parent Class."<<endl;}
4 };
5
6 class Child : public Parent {
7
8 };
9
10 class GrandChild : public Child{
11
12 };
13
14 int main(){
15     GrandChild GC;  GC.message();
16     return 0;
17 }
```

Parent Class.

Example 4

```
1 class Parent {
2     public:
3         void message(){cout<<"Parent Class."<<endl;}
4 };
5
6 class Child : public Parent {
7     public:
8         void message(){cout<<"Child Class."<<endl;}
9 };
10
11 class GrandChild : public Child{
12     public:
13         void message(){Child::message();}
14 };
15
16 int main(){
17     GrandChild GC; GC.message();
18     return 0;
19 }
```

Child Class.

Example 5

```
1 class Parent {
2     public:
3         void message(){cout<<"Parent Class."<<endl;}
4 };
5
6 class Child : public Parent {
7     public:
8         void message(){cout<<"Child Class."<<endl;}
9 };
10
11 class GrandChild : public Child{
12     public:
13         void message(){Parent::message();}
14 };
15
16 int main(){
17     GrandChild GC; GC.message();
18     return 0;
19 }
```

Parent Class.

Example 6

```
1 class Parent {
2     public:
3         void message(){cout<<"Parent Class."<<endl;}
4 };
5
6 class Child : public Parent {
7     public:
8         void message(){Parent::message();}
9 };
10
11 class GrandChild : public Child{
12
13 };
14
15 int main(){
16     GrandChild GC; GC.message();
17     return 0;
18 }
```

Parent Class.

Example 7

```
1 class Parent {
2     public:
3         void message(){cout<<"Parent Class."<<endl;}
4 };
5
6 class Child : public Parent {
7     public:
8         void message(){cout<<"Child Class."<<endl;}
9 };
10
11 class GrandChild : public Child{
12     public:
13         void message(){cout<<"Grand Child Class."<<endl;}
14 };
15
16 int main(){
17
18     GrandChild GC; GC.message();
19     GC.Child::message(); GC.Parent::message();
20     return 0;
21 }
```

Grand Child Class.
Child Class.
Parent Class.

Example 1

```
1 class Parent {
2     public:
3         Parent(){cout<<"Parent Constructor."<<endl;}
4         ~Parent(){cout<<"Parent Destructor."<<endl;}
5 };
6
7 class Child : public Parent {
8     public:
9         Child(){cout<<"Child Constructor."<<endl;}
10        ~Child(){cout<<"Child Destructor."<<endl;}
11 };
12
13 class GrandChild : public Child{
14     public:
15         GrandChild(){cout<<"GrandChild Constructor."<<endl;}
16         ~GrandChild(){cout<<"GrandChild Destructor."<<endl;}
17 };
18 int main(){
19     GrandChild GC;
20     return 0;
21 }
```

Parent Constructor.
Child Constructor.
GrandChild Constructor.
GrandChild Destructor.
Child Destructor.
Parent Destructor.

Example 2

```
1 class Parent {
2     public:
3         Parent (int a){cout<<"Parent:"<<a<<endl;}
4 };
5
6 class Child : public Parent {
7     public:
8         Child(int a, int b): Parent (a)
9             {cout<<"Child:"<<b<<endl;}
10 };
11
12 class GrandChild : public Child{
13     public:
14         GrandChild(int a, int b, int c): Child (a,b)
15             {cout<<"Grand Child:"<<c<<endl;}
16 };
17
18 int main(){
19     GrandChild GC(40,74,6);
20     return 0;
21 }
```

```
Parent:40
Child:74
Grand Child:6
```

Example 3

```
1 class Parent {
2     public:
3         Parent (int a){cout<<"Parent:"<<a<<endl;}
4 };
5
6 class Child : public Parent {
7     public:
8         Child(int a, int b)
9             {cout<<"Child:"<<b<<endl;}
10 };
11
12 class GrandChild : public Child{
13     public: // Following is incorrect (NO direct communication.)
14         GrandChild(int a,int b,int c):Child (a,b),Parent (c)
15             {cout<<"Grand Child:"<<c<<endl;}
16 };
17
18 int main(){
19     GrandChild GC(40,74,6);
20     return 0;
21 }
```

Scenario 8.2

```
1 class Person {
2     string name;
3     int age;
4
5     public:
6         Person(string name, int age) : name(name), age(age) {}
7
8         void setName(string name) { this->name = name; }
9         void setAge(int age) { this->age = age; }
10
11        string getName() const { return name; }
12        int getAge() const { return age; }
13
14        void printInfo() const {
15            cout << "Name: " << name << "\n";
16            cout << "Age: " << age << "\n";
17        }
18 };
```

Scenario 8.2 (Cont.)

```
1 class Student : public Person {
2     int studentId;
3     string grade;
4
5     public:
6         Student(string name, int age, int Id, string grade)
7             : Person(name, age), studentId(Id), grade(grade) {}
8
9         void setStudentId(int Id) { studentId = Id; }
10        void setGrade(string grade) { this->grade = grade; }
11
12        int getStudentId() const { return studentId; }
13        string getGrade() const { return grade; }
14
15        void printInfo() const {
16            Person::printInfo(); // Call Person's method
17            cout << "Student ID: " << studentId << "\n";
18            cout << "Grade: " << grade << "\n";
19        }
20    };
```

Scenario 8.2 (Cont.)

```
1 class GraduateStudent : public Student {
2     string researchTopic;
3     string advisorName;
4
5 public:
6     GraduateStudent(string name,int age,int Id,string grade,
7         string topic,string advisor):Student(name,age,Id,
8         grade),researchTopic(topic),advisorName(advisor) {}
9
10
11 void setTopic(string topic){researchTopic = topic;}
12 void setAdvisor(string advisor){advisorName = advisor;}
13
14 string getTopic() const {return researchTopic; }
15 string getAdvisor() const {return advisorName; }
16
17 void printInfo() const {
18     Student::printInfo(); // Call base class method
19     cout<<"Research Topic: " << researchTopic << "\n";
20     cout<<"Advisor Name: " << advisorName << "\n";
21 }
22 };
```

Scenario 8.2 (Cont.)

```
1 int main() {  
2  
3     GraduateStudent GS("Abbas Khalid", 25, 8288151, "A+", "  
        Channel Codes", "Prof. Bahram Honary");  
4     GS.printInfo();  
5  
6     return 0;  
7 }
```

```
Name: Abbas Khalid  
Age: 25  
Student ID: 8288151  
Grade: A+  
Research Topic: Channel Codes  
Advisor Name: Prof. Bahram Honary
```

Conclusions

In this lecture we discussed about

- Inheritance
- Protected Members
- Public, Protected, and Private Inheritance
- Constructors and Destructors in Inheritance
- Multi-level Inheritance

In the next lecture we will discuss more about

- Inheritance